INTERVIEW PREPARATION GUIDE

100

React (JS) Interview Questions

With Concise Model Answers for Quick Revision

Prepared by **Pushkar Singh**

August 2026

A QUICK-REFERENCE INTERVIEW COMPANION

CONTENTS

Table of Contents

01	React Fundamentals	**3**
02	JSX & Components	**5**
03	Props & State	**7**
04	Hooks	**9**
05	Component Lifecycle	**11**
06	Event Handling & Forms	**13**
07	React Router	**15**
08	State Management (Context/Redux)	**17**
09	Performance Optimization	**19**
10	Testing, Tooling & Miscellaneous	**21**

Ten questions per chapter — 100 questions in all

CHAPTER 01

React Fundamentals

001 What is React?

React is a JavaScript library for building user interfaces, developed by Facebook. It uses a component-based architecture and a virtual DOM for efficient updates. It’s mainly used for building single-page applications.

002 What is the Virtual DOM?

The Virtual DOM is a lightweight, in-memory representation of the actual DOM. React updates it first, compares it with the previous version (diffing), and then updates only the changed parts in the real DOM, making updates faster.

003 What is JSX?

JSX (JavaScript XML) is a syntax extension that lets you write HTML-like code within JavaScript. It gets compiled into React.createElement() calls by tools like Babel, making component structure easier to read and write.

004 What is a React component?

A component is a reusable, independent piece of UI that can accept inputs (props) and return React elements describing what should appear on screen. Components can be functional or class-based and composed together.

005 What is the difference between functional and class components?

Functional components are plain functions that return JSX and use hooks for state/lifecycle features. Class components extend React.Component and use methods like render(). Functional components with hooks are now the preferred approach.

006 What is the difference between React and ReactDOM?

React is the core library providing component logic, state, and the Virtual DOM. ReactDOM is a separate package used to render React components into the actual browser DOM. React can also target other renderers, like React Native.

007 What is a key prop and why is it important?

A key is a special string attribute used when rendering lists, helping React identify which items changed, were added, or removed. Keys should be stable and unique; using array indexes can cause bugs when list order changes.

008 What is the difference between controlled and uncontrolled components?

Controlled components have their form data managed by React state, with value and onChange handlers. Uncontrolled components store state internally in the DOM, accessed via refs, giving React less control over the data.

009 What is React Fiber?

React Fiber is the reconciliation engine introduced in React 16 that rewrote the core rendering algorithm. It enables incremental rendering, allowing React to pause, prioritize, and resume work for better responsiveness.

010 Why must JSX return a single root element?

JSX compiles to a single React.createElement() call, so an expression must return one parent element. Multiple siblings can be wrapped in a or a React Fragment (<> </>) without adding extra DOM nodes.

CHAPTER 02

JSX & Components

011 How do you pass data between components?

Data flows from parent to child using props. To pass data from child to parent, the parent passes a callback function as a prop that the child calls with the data. For siblings, state is typically lifted to a common parent.

012 What is prop drilling?

Prop drilling is passing data through multiple layers of nested components via props, even when intermediate components don’t need it themselves. It can make code harder to maintain; Context API helps avoid it.

013 What are React Fragments?

Fragments let you group a list of children without adding extra nodes to the DOM. They’re written as <React.Fragment> or the shorthand <> </>, useful when a component must return multiple elements without a wrapper div.

014 What is the difference between state and props?

Props are read-only data passed from a parent to configure a child. State is data managed within a component that can change over time and trigger re-renders. Props are immutable from the child’s view; state is mutable.

015 What is component composition?

Component composition is building complex UIs by combining smaller, reusable components, often passing components as children or props. It’s React’s preferred alternative to inheritance for code reuse.

016 What are higher-order components (HOCs)?

A higher-order component is a function that takes a component and returns a new enhanced component with additional props or behavior. HOCs are used for cross-cutting concerns like authentication or logging.

017 What are render props?

Render props is a pattern where a component receives a function as a prop that returns JSX, letting it share logic dynamically. The component calls this function inside its output instead of hard-coding what to render.

018 What is the children prop in React?

The children prop is a special prop holding whatever is nested between a component’s opening and closing tags. It allows components to wrap and render arbitrary content, common in layout or modal components.

019 What is PropTypes used for?

PropTypes is a library for runtime type-checking of props passed to components. It warns in the console when props don’t match expected types; TypeScript is now often used instead for compile-time checking.

020 What is the significance of defaultProps?

defaultProps lets you define default values for props in case they aren’t explicitly passed by the parent. It ensures components behave predictably; function default parameters are often used instead in modern React.

CHAPTER 03

Props & State

021 What is state in React?

State is a built-in object that stores data that can change over a component's lifetime. When state changes, React re-renders the component to reflect the new data; state is local unless lifted up or shared via context.

022 How do you update state correctly?

State should be updated using the setState function (class components) or the setter from useState (functional components), never by mutating it directly. The functional updater form (prev => ...) is recommended when new state depends on old state.

023 Why shouldn't you mutate state directly?

Directly mutating state bypasses React's change detection, so the component may not re-render or may behave unpredictably. React relies on new state references to detect changes efficiently, so always create a copy when updating.

024 What is lifting state up?

Lifting state up means moving shared state to the closest common ancestor of components that need it. The parent then passes the state and updater functions down as props, keeping sibling components in sync.

025 What is the difference between local state and global state?

Local state is managed within a single component using useState and isn't accessible elsewhere. Global state is shared across multiple components, often via Context API or libraries like Redux, suiting data needed app-wide.

026 How does React batch state updates?

React groups multiple state updates within the same event handler into a single re-render for performance. Since React 18, this automatic batching also applies to updates inside promises, timeouts, and native event handlers.

027 What happens when you call a state setter multiple times in one function?

React batches these calls into a single update cycle rather than re-rendering after each. If updates depend on previous state, the functional form ensures each builds correctly on the last; otherwise some may use stale state.

028 What is derived state in React?

Derived state is state calculated from props or other state values rather than stored independently. It’s usually better to compute derived values during rendering instead of storing them, to avoid synchronization bugs.

029 What is the initial state in useState?

The initial state is the value passed as the argument to useState(), e.g., useState(0). It’s only used on the component’s first render; a function can be passed for expensive initial computations (lazy initialization).

030 Can you use multiple useState calls in one component?

Yes, React allows and encourages multiple useState calls to manage separate, independent pieces of state within a single component, keeping state organized compared to one large state object.

CHAPTER 04

Hooks

031 What are React Hooks?

Hooks are functions that let functional components use state, lifecycle features, and other React capabilities without writing a class. Common hooks include useState, useEffect, and useContext, introduced in React 16.8.

032 What is useEffect used for?

useEffect lets you perform side effects in functional components, such as data fetching, subscriptions, or DOM changes. It runs after render and can optionally clean up via a returned function, controlled by its dependency array.

033 What is the dependency array in useEffect?

The dependency array is the second argument to useEffect, listing values the effect depends on. React re-runs the effect only when one of these values changes; an empty array means it runs once after the initial render.

034 What is useContext used for?

useContext lets a component read the current value of a Context object without wrapping it in a Consumer. It simplifies accessing shared data like themes or authentication state, used with a Context from createContext.

035 What is useRef used for?

useRef returns a mutable ref object that persists across renders without causing re-renders when changed. It’s commonly used to access DOM elements directly or store mutable values that don’t affect rendering.

036 What is useMemo used for?

useMemo memoizes the result of an expensive computation, recalculating it only when its dependencies change. This avoids unnecessary recalculations on every render, and should be used only when there’s a genuine performance need.

037 What is useCallback used for?

useCallback memoizes a function definition, returning the same reference between renders unless its dependencies change. This prevents unnecessary re-renders of child components relying on reference equality, like those wrapped in React.memo.

038 What is the difference between useMemo and useCallback?

useMemo memoizes and returns a computed value, while useCallback memoizes and returns a function itself. Both take a dependency array; useCallback(fn, deps) is essentially useMemo(() => fn, deps).

039 What are the rules of hooks?

Hooks must only be called at the top level of a function component or custom hook, never inside loops, conditions, or nested functions, and only from React functions. This ensures hooks run in the same order every render.

040 What is a custom hook?

A custom hook is a reusable function whose name starts with “use” that can call other hooks to encapsulate and share stateful logic between components. It shares logic patterns, not state itself, e.g., useFetch or useLocalStorage.

CHAPTER 05

Component Lifecycle

041 What are the phases of a React component’s lifecycle?

The three main phases are mounting (component created and inserted into the DOM), updating (re-renders due to changed props/state), and unmounting (component removed from the DOM), each with related methods or hooks.

042 What is componentDidMount used for?

componentDidMount is a class lifecycle method called once immediately after a component first renders to the DOM. It’s commonly used for data fetching or subscriptions; its hook equivalent is useEffect with an empty dependency array.

043 What is componentDidUpdate used for?

componentDidUpdate is called after a component re-renders due to changed props or state, excluding the initial render. It’s often used to react to specific changes, like refetching data; its hook equivalent uses relevant dependencies.

044 What is componentWillUnmount used for?

componentWillUnmount is called right before a component is removed from the DOM, used for cleanup like clearing timers or removing listeners. Its hook equivalent is the cleanup function returned from useEffect.

045 How do you mimic lifecycle methods using hooks?

useEffect(fn, []) mimics componentDidMount, useEffect(fn, [dep]) mimics componentDidUpdate for specific dependencies, and returning a cleanup function mimics componentWillUnmount.

046 What is the difference between mounting and updating?

Mounting refers to a component being created and inserted into the DOM for the first time. Updating refers to a component re-rendering due to changes in props, state, or context after it has already mounted.

047 What is getDerivedStateFromProps?

getDerivedStateFromProps is a static class lifecycle method invoked before every render to update state based on prop changes. It’s rarely needed and generally discouraged in favor of computing derived values during render.

048 What is shouldComponentUpdate used for?

shouldComponentUpdate is a class lifecycle method letting you control whether a component re-renders, based on prop/state comparisons. It’s used for performance optimization; React.memo serves a similar purpose functionally.

049 What is React.PureComponent?

PureComponent is a base class that automatically implements shouldComponentUpdate with a shallow comparison of props and state, preventing unnecessary re-renders. Its functional equivalent is wrapping a component with React.memo.

050 What is the cleanup function in useEffect?

The cleanup function is an optional function returned from useEffect that React runs before the component unmounts or before the effect re-runs, used to clean up subscriptions, timers, or listeners and prevent memory leaks.

CHAPTER 06

Event Handling & Forms

051 How does event handling work in React?

React uses SyntheticEvent, a cross-browser wrapper around native DOM events, providing consistent behavior. Handlers are passed as camelCase props like onClick; React attaches listeners efficiently and delegates events internally.

052 What is a SyntheticEvent?

A SyntheticEvent is React’s wrapper around the browser’s native event object, normalizing properties and behavior across browsers, with the same interface as native events including stopPropagation() and preventDefault().

053 How do you handle form inputs in React?

Form inputs are typically handled as controlled components, tying the input’s value to state and updating it via onChange, giving React full control. Uncontrolled inputs use refs to access the DOM value directly instead.

054 How do you prevent a form’s default submission behavior?

You call event.preventDefault() inside the form’s onSubmit handler to stop the browser’s default page reload, letting you handle submission logic manually, such as validation or an API call.

055 What is the difference between onChange in React and the native change event?

React’s onChange fires on every keystroke or input change, similar to the native “input” event, rather than only on blur like the native “change” event, giving more immediate feedback for controlled components.

056 How do you pass arguments to an event handler?

You can pass arguments using an arrow function or bind, like onClick={() => handleClick(id)}, letting you supply custom parameters beyond the default event object when the handler runs.

057 What is event delegation in React?

Event delegation is a technique where React attaches a single event listener at the root of the app instead of on individual elements. Events bubble up and are handled centrally, improving performance and reducing memory use.

058 How do you handle multiple form inputs with a single handler?

You can use one handler function that reads the input’s name attribute via event.target.name to determine which state field to update, often using computed property names like setState({ [name]: value }).

059 What is form validation in React and how is it typically done?

Form validation checks user input against required rules before submission. It can be done manually with conditional logic and state, or using libraries like Formik, React Hook Form, or Yup for complex validation schemas.

060 What is the difference between value and defaultValue in form inputs?

value is used for controlled components, where React fully manages the input’s current value via state. defaultValue is used for uncontrolled components, setting only the initial value while the DOM manages changes.

CHAPTER 07

React Router

061 What is React Router?

React Router is a standard library for handling client-side routing and navigation in React single-page applications, rendering different components based on the URL without a full page reload.

062 What is the difference between BrowserRouter and HashRouter?

BrowserRouter uses the HTML5 History API for clean URLs (e.g., /about), requiring server configuration for direct access. HashRouter uses the URL hash (e.g., /#/about) and works without special server setup, but with less clean URLs.

063 What is the purpose of the Link component?

The Link component renders an anchor tag that enables client-side navigation between routes without a full page reload, preserving the single-page application experience and application state.

064 How do you access URL parameters in React Router?

URL parameters (like /:id in the route path) are accessed using the useParams hook, which returns an object with the matched parameters, commonly used to fetch data specific to a dynamic route.

065 What is nested routing?

Nested routing lets you render child routes within a parent route's component, typically using an <Outlet /> placeholder in the parent to display the matched child route — useful for persistent layouts.

066 How do you programmatically navigate in React Router?

You use the useNavigate hook (React Router v6+), calling navigate('/path') to redirect the user programmatically, such as after a successful form submission or login.

067 What is a protected/private route?

A protected route checks a condition, like user authentication, before rendering its content, redirecting unauthorized users elsewhere. It's typically a wrapper component that conditionally renders the route or a redirect.

068 What does the useLocation hook do?

useLocation returns an object representing the current URL location, including the pathname, search query string, and state, useful for reading query parameters or route-based logic.

069 What is the difference between Route and Switch/Routes?

Route defines a single path-to-component mapping. Switch (v5) or Routes (v6) wraps multiple Route elements and renders only the first matching route, preventing overlapping routes from rendering together.

070 How do you handle a 404 / not-found page in React Router?

You define a catch-all route using a wildcard path (path="*") placed as the last Route in your list. Since routes are matched in order or by specificity, this renders whenever no other route matches.

CHAPTER 08

State Management (Context/Redux)

071 What is the Context API?

The Context API is a built-in React feature for sharing data like themes or user info across the component tree without manually passing props at every level, using createContext, a Provider, and useContext.

072 What problem does Context API solve?

Context API solves prop drilling, where data must be passed through many intermediate components that don’t need it themselves, just to reach a deeply nested child, by sharing values directly with components that need them.

073 What is Redux?

Redux is a predictable state management library that stores an application’s entire state in a single centralized store. Updates happen through pure functions called reducers, triggered by dispatched actions.

074 What are the core principles of Redux?

Redux follows three principles: a single source of truth (one store), state is read-only (changed only via dispatched actions), and changes are made using pure reducer functions, making updates predictable and traceable.

075 What is an action in Redux?

An action is a plain JavaScript object describing what happened, typically containing a type field and optional payload data. Actions are dispatched to the store, and reducers use them to update state.

076 What is a reducer in Redux?

A reducer is a pure function that takes the current state and an action, returning a new state based on the action type. Reducers must not mutate existing state directly and should have no side effects.

077 What is the difference between Redux and Context API?

Context API is simpler and best suited for low-frequency updates like themes. Redux is a more structured library with dev tools, middleware support, and better performance for complex, frequently-changing state.

078 What is middleware in Redux?

Middleware sits between dispatching an action and the reducer, letting you intercept, modify, log, or handle side effects like asynchronous API calls. Popular examples include redux-thunk and redux-saga.

079 What is Redux Toolkit?

Redux Toolkit is the official, recommended way to write Redux logic, simplifying store setup and reducer creation using simpler APIs like createSlice and configureStore, reducing boilerplate compared to classic Redux.

080 What are alternatives to Redux for state management?

Popular alternatives include Context API combined with useReducer for simpler apps, and libraries like Zustand, Recoil, Jotai, or MobX, offering lighter-weight or differently structured global state approaches.

CHAPTER 09

Performance Optimization

081 What is React.memo used for?

React.memo is a higher-order component that memoizes a functional component, preventing re-renders if its props haven’t changed (shallow comparison), useful for components that render often with the same props.

082 What causes unnecessary re-renders in React?

Common causes include passing new object/array/function references as props on every render, updating widely-consumed state or context, and not memoizing expensive child components.

083 What is code splitting in React?

Code splitting breaks a large application bundle into smaller chunks loaded on demand rather than all at once, reducing initial load time. React supports this via React.lazy and dynamic imports.

084 What is React.lazy used for?

React.lazy lets you dynamically import a component so it’s only loaded when actually rendered, enabling code splitting at the component level, typically used together with the Suspense component.

085 What is the Suspense component?

Suspense lets you display a fallback UI, like a loading spinner, while waiting for an asynchronous operation such as a lazily-loaded component to finish, wrapping components that aren’t immediately ready.

086 What is windowing/virtualization in React lists?

Windowing (virtualization) renders only the visible portion of a large list rather than all items at once, significantly improving performance. Libraries like react-window and react-virtualized implement this.

087 Why should you avoid creating functions inline in JSX for performance-sensitive components?

Defining functions inline creates a new function reference on every render, which can cause memoized child components to re-render unnecessarily since their props appear changed; useCallback keeps a stable reference.

088 What is the key prop’s role in list performance?

Keys help React efficiently identify which list items changed, were added, or removed during reconciliation, avoiding unnecessary DOM recreation. Stable, unique keys improve rendering performance and correctness.

089 How can you profile performance issues in a React app?

The React DevTools Profiler lets you record rendering performance, showing which components rendered, how long they took, and why, helping pinpoint unnecessary renders or slow components needing optimization.

090 What is the difference between React.memo and useMemo?

React.memo memoizes an entire component to prevent re-rendering when props are unchanged. useMemo memoizes a computed value within a component to avoid expensive recalculations, solving similar problems at different levels.

CHAPTER 10

Testing, Tooling & Miscellaneous

091 What is Create React App / Vite used for?

These tools scaffold a new React project with a preconfigured build setup, including bundling and a dev server, so developers don’t need to configure Webpack/Babel manually. Vite is known for fast startup and hot reloading.

092 What testing libraries are commonly used with React?

Jest is commonly used as the test runner and assertion library, while React Testing Library tests components by simulating user interactions and checking rendered output, enabling unit and integration testing.

093 What is the philosophy behind React Testing Library?

React Testing Library encourages testing components the way users interact with them, focusing on rendered output and behavior rather than internal implementation details, leading to more resilient tests.

094 What is a snapshot test?

A snapshot test captures a component’s rendered output and saves it as a reference file. Future test runs compare the current output against this snapshot, flagging unexpected changes for review.

095 What is the difference between shallow and full (deep) rendering in testing?

Shallow rendering renders only the component itself without its children, useful for isolated unit tests. Full/deep rendering renders the component with all children, useful for integration-style tests.

096 What is StrictMode in React?

StrictMode is a development-only tool that helps detect potential problems, such as unsafe lifecycle methods and side effects that aren’t idempotent, by intentionally double-invoking certain functions; it has no effect in production.

097 What is the difference between npm and Webpack in a React project?

npm (or yarn/pnpm) is a package manager used to install and manage dependencies. Webpack is a module bundler that combines JavaScript, CSS, and other assets into optimized bundles for the browser.

098 What is server-side rendering (SSR) in the context of React?

SSR renders React components to HTML on the server before sending them to the browser, rather than rendering everything client-side, improving initial load performance and SEO. Next.js provides built-in SSR support.

099 What is hydration in React?

Hydration is the process where React attaches event listeners and takes over server-rendered HTML on the client, making it interactive without re-rendering the entire markup from scratch.

100 What is the difference between React and Next.js?

React is a library focused purely on building UI components and managing rendering. Next.js is a full framework built on React that adds server-side rendering, static generation, file-based routing, and API routes.